Compiladores 2025/26

Análise Sintática Ascendente

Raul Barbosa

Departamento de Engenharia Informática
Faculdade de Ciências e Tecnologia
Universidade de Coimbra

· U C ·

Análise sintática

Análise sintática ascendente

- A construção da árvore é efetuada a partir das folhas (*tokens*) na direção da raiz (símbolo inicial).
- A cada passo, o analisador encontra uma sub-sequência na parte superior da árvore (parcialmente construída) que seja igual ao lado direito de alguma produção;
- Nesse ponto, cria um novo nó para o lado esquerdo dessa produção e associa-o aos seus filhos (os símbolos encontrados).
- A análise sintática ascendente efetua uma derivação pela direita por ordem inversa.

Análise por deslocamento e redução (*shift-reduce parsing*)

O analisador divide a entrada em duas sub-sequências $\alpha X . \gamma$

- A sub-sequência esquerda αX contém não-terminais e terminais parcialmente analisados. Representada por uma pilha.
- A sub-sequência direita γ só contém terminais por analisar.

Ações de deslocamento (*shift*)

- Lê um terminal da entrada e desloca-o para a pilha.

Ações de redução (*reduce*)

- Aplica uma redução (produção inversa) ao topo da pilha.
- Remove da pilha o lado direito completo de uma produção.
- Coloca na pilha o lado esquerdo dessa produção.

Exemplo: gramática dos parêntesis

Gramática representativa de linguagens que usem () {} <> []

$$
\begin{array}{lcl}
L & \rightarrow & L \;\; P \\
 & | & P \\
P & \rightarrow & (\;\; L \;\;) \\
 & | & (\;\;)
\end{array}
$$

Sequência de estados do analisador para a entrada ()

Pilha	Entrada	Ação
	()$	desloca
(	)$	desloca
()	$	reduz $P \rightarrow (\;\;)$
P	$	reduz $L \rightarrow P$
L	$	aceita

Termina com o símbolo inicial no topo da pilha e $ como *lookahead*.

Decidir quando deslocar e quando reduzir

Só se reduz se o resultado puder ser reduzido ao símbolo inicial

A analisador repete um processo simples:

- Repetidamente desloca símbolos terminais para uma pilha.
- Quando encontra o lado direito de determinada produção $A \rightarrow \alpha$ no topo da pilha, reduz α a A.

O analisador LR(1)

- Os analisadores sintáticos LR(1) são particularmente eficientes a reconhecer na pilha lados direitos de produções (*"handles"*).
- **L**eft-to-right input scan, producing a **R**ightmost derivation in reverse, using **1** lookahead token.

Reconhecer *"handles"* (lados direitos de produções)

- O analisador só vê a pilha (e um símbolo de antecipação).
- Definição: α é um *prefixo viável* se $\alpha.\gamma$ for um estado válido de um analisador ascendente.
- É um *prefixo* porque não se extende para lá da *"handle"*.
- É *viável* porque contém os primeiros símbolos da *"handle"*.

O conjunto de prefixos viáveis é uma linguagem regular.

- Os lados direitos das produções têm número finito de símbolos.
- Podem ser reconhecidos por um autómato finito.

O analisador sintático LR(1)

Um analisador LR(1) utiliza um autómato finito determinístico codificado na forma de tabela:

- A tabela está dividida em duas partes: Action e Goto.
- Esta tabela codifica todas as sequências de estados que podem surgir numa derivação pela direita por ordem inversa.
- É construída de forma algorítmica a partir da gramática.

Itens LR(1)

Um item LR(1) é da forma $[A \to \alpha.\beta, t]$ e consiste em:

- Uma produção com um . indica que α está no topo da pilha
- Na cabeça de leitura está uma sequência derivável de βt
- O *lookahead* t terá de ser encontrado após processar β
- Quando $\alpha\beta$ estiver no topo da pilha, reduz-se com $A \to \alpha\beta$ se e só se o símbolo seguinte for t

Gramática aumentada com uma produção $start \to L\,\$$

$$
\begin{array}{lcl}
start & \to & L\ \$ \\
L & \to & L\ P \\
 & | & P \\
P & \to & (\ L\ ) \\
 & | & (\ )
\end{array}
$$

Itens: $[start \to .L, \$]$ $[start \to L., \$]$ $[L \to .LP, \$]$ $[L \to L.P, \$]$ $\cdots$

Closure(I) – fechar conjuntos de itens

Partindo do conjunto I, acrescentar-lhe novos itens quando o ponto estiver à esquerda de um não-terminal.

Se o analisador estiver num estado contendo $[A \to \alpha.X\beta, z]$ então espera-se ver X na pilha ou algo que se reduza a X.

Closure(I) =
  repeat
    for each item $[A \to \alpha.X\beta, z]$ in I
      for each production $X \to \gamma$
        for each $w \in$ FIRST(βz)
          $I \leftarrow I \cup \{[X \to .\gamma, w]\}$
  until I does not change
  return I

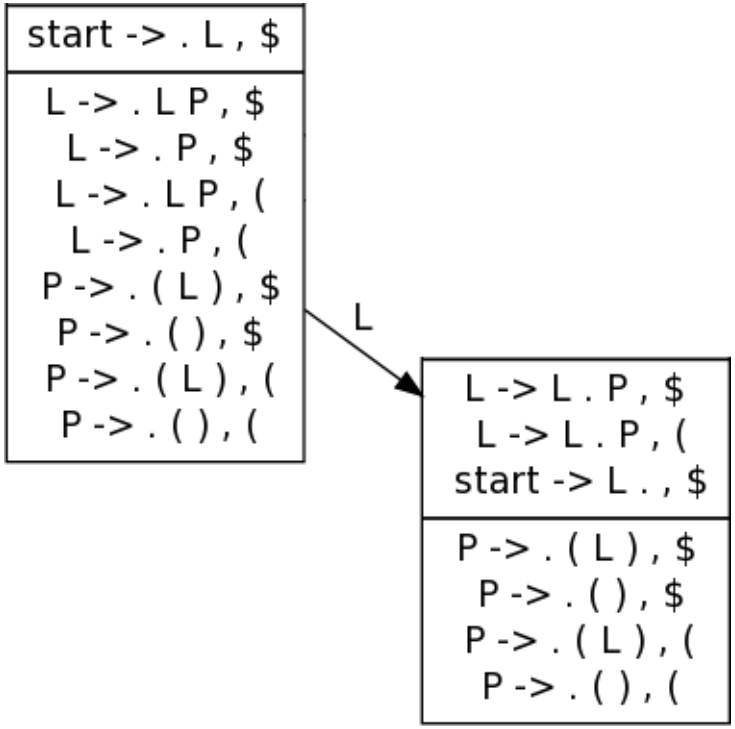
start -> . L , $
L -> . L P , $
L -> . P , $
L -> . L P , (
L -> . P , (
P -> . (L) , $
P -> . ( ) , $
P -> . (L) , (
P -> . ( ) , (

Goto(I, X) – transitar para um novo estado

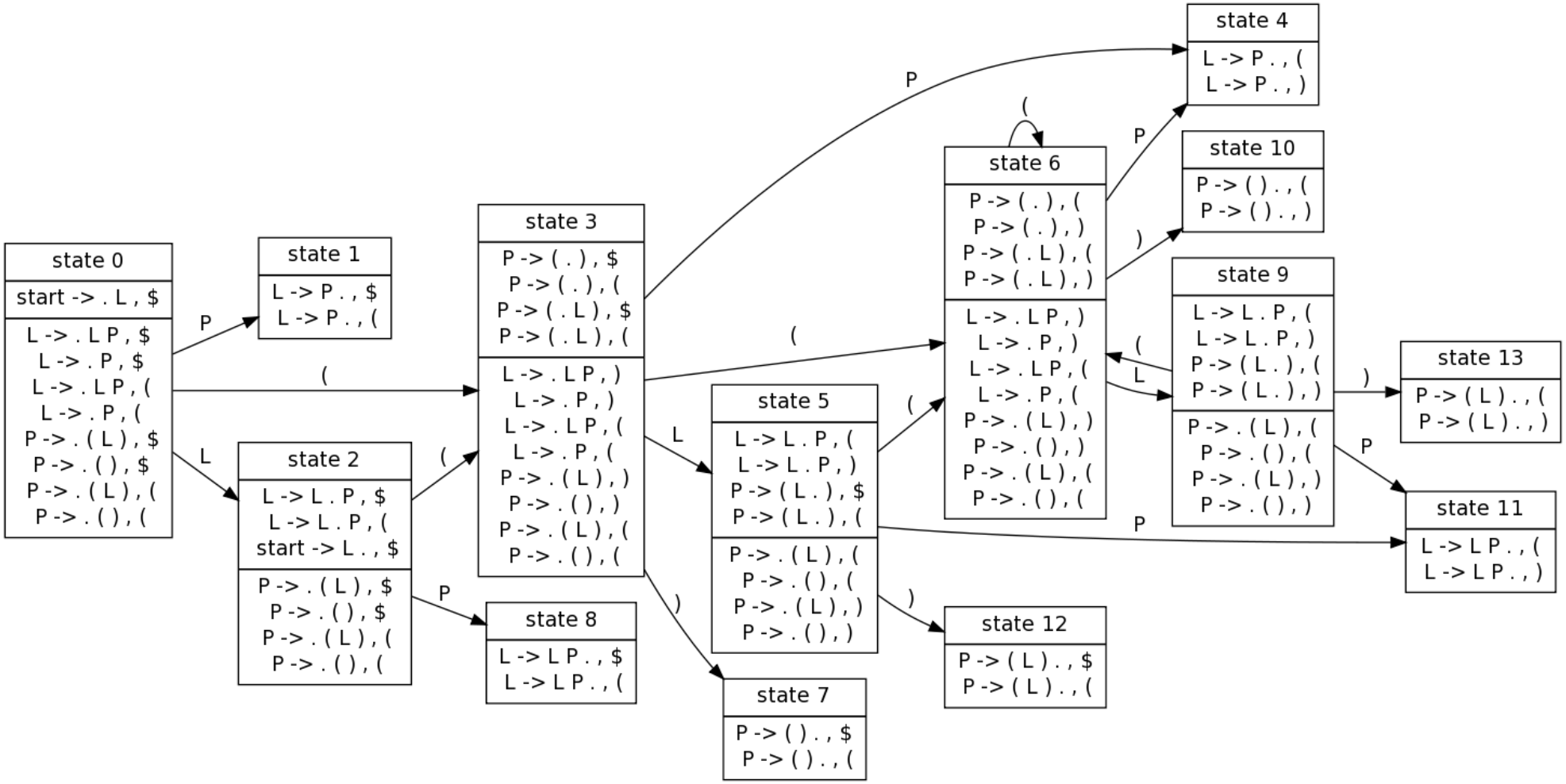
Mover, em todos os itens possíveis, o ponto para a direita de X

Goto(I, X) faz transitar o analisador do atual conjunto de itens LR(1) para outro, após reconhecer o terminal ou não-terminal X.

Goto(I, X) =
  J $\leftarrow \{\}$
  for each item $[A \rightarrow \alpha.X\beta, z]$ in I
    $J \leftarrow J \cup \{[A \rightarrow \alpha X.\beta, z]\}$
  return Closure(J)



Autómato LR(1) – coleção canónica de conjuntos de itens



- Estado inicial: Closure({ [start $\rightarrow$ .L, \$] })
- Restantes: Goto(S_i, x) para cada estado S_i e cada símbolo x

Construção da tabela do analisador LR(1)

1. Para cada aresta $I \xrightarrow{X} J$:
   - Se X é terminal, coloca-se shift J na posição (I,X)
   - Se X é não-terminal, coloca-se goto J na posição (I,X)
2. Para cada estado I contendo um item $[start \rightarrow S., \$]$ coloca-se accept em (I,$) – termina aí a análise.
3. Para cada estado I contendo um item $[\alpha \rightarrow \gamma., t]$ coloca-se reduce n em (I, t) onde a produção $\alpha \rightarrow \gamma$ tem o número n.

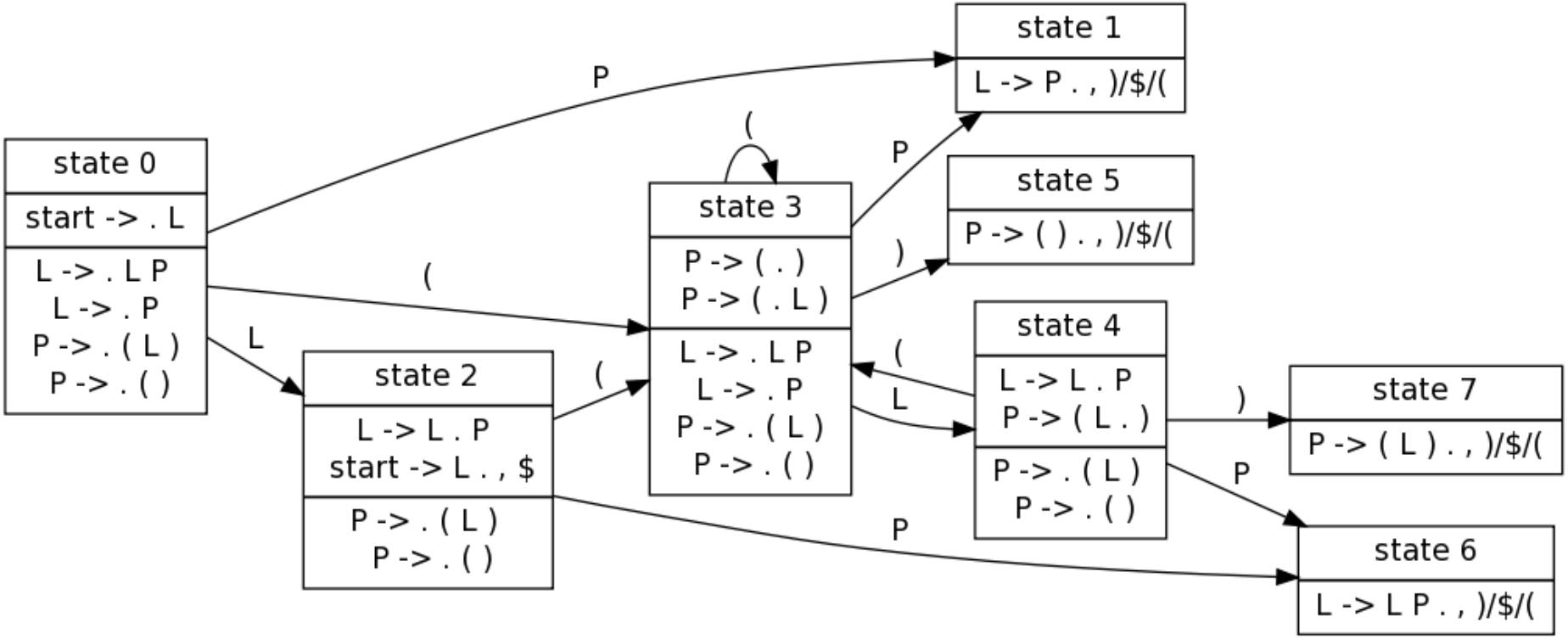
Tabela LR(1)

	$	)	(	L	P
0			s3	g2	g1
1	r(L → P)		r(L → P)		
2	accept		s3		g8
3		s7	s6	g5	g4
4		r(L → P)	r(L → P)		
5		s12	s6		g11
6		s10	s6	g9	g4
7	r(P → ( ))		r(P → ( ))		
8	r(L → L P)		r(L → L P)		
9		s13	s6		g11
10		r(P → ( ))	r(P → ( ))		
11		r(L → L P)	r(L → L P)		
12	r(P → (L))		r(P → (L))		
13		r(P → (L))	r(P → (L))		

O analisador LALR(1) – Look-Ahead LR(1)

- Um analisador LALR(1) obtém-se a partir do analisador LR(1) fundindo os estados que sejam idênticos exceto nos símbolos de lookahead.
- O número de estados é assim reduzido significativamente, à custa de uma pequena perda de generalidade.
- Analisadores sintáticos LALR(1) são uma escolha muito frequente para diversas linguagens de programação.

Autómato LALR(1)



Exercício

Gramática:

$$
\begin{array}{lcl}
start & \rightarrow & S \ \$ \\
S & \rightarrow & id \ L \ ; \\
L & \rightarrow & L \ S \\
 & | & \epsilon
\end{array}
$$

Algumas frases geradas por esta gramática:

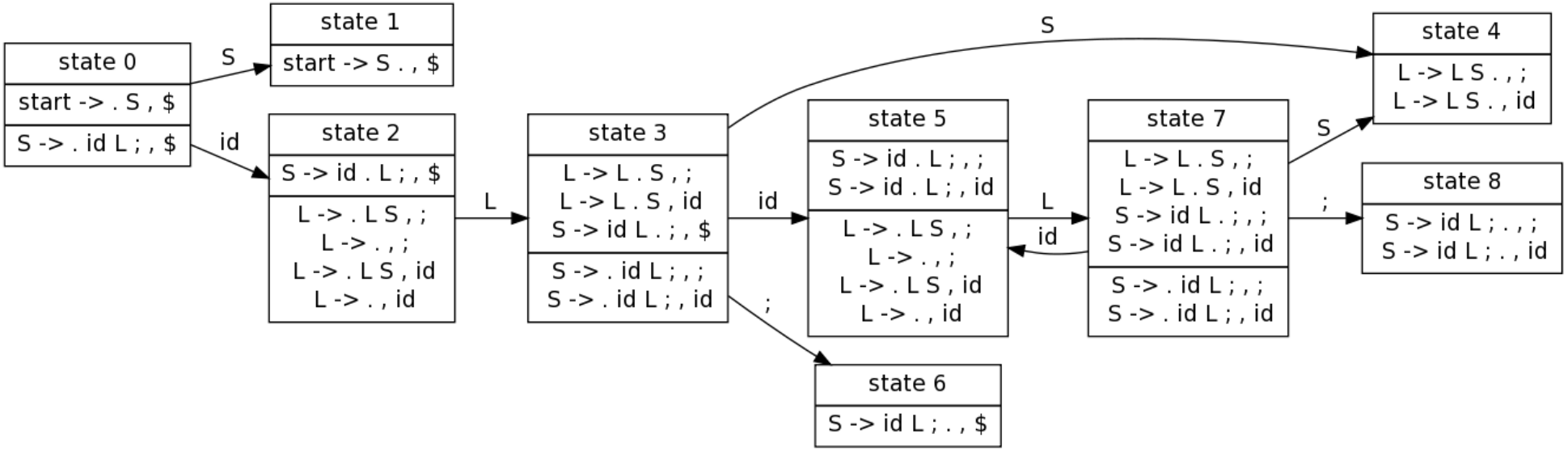
id ; id id ; ; id id ; id ; ; id id id ; id ; ; ;

Mostra-se que esta gramática é LR(1) apresentando o autómato e a tabela do respetivo analisador sintático.

Autómato e tabela LR(1)

$$
\begin{aligned}
start &\rightarrow S\ \$ \\
S &\rightarrow id\ L\ ; \\
L &\rightarrow L\ S \\
&\mid\ \epsilon
\end{aligned}
$$

	$	;	id	S	L
0			s2	g1	
1	accept				
2		r(L → ε)	r(L → ε)		g3
3		s6	s5	g4	
4		r(L → L S)	r(L → L S)		
5		r(L → ε)	r(L → ε)		g7
6	r(S → id L ;)				
7		s8	s5	g4	
8		r(S → id L ;)	r(S → id L ;)		



Compiladores 2025/26

Análise Sintática Ascendente

Raul Barbosa

Departamento de Engenharia Informática
Faculdade de Ciências e Tecnologia
Universidade de Coimbra

· U C ·